

LAPORAN ALGORITMA DAN STRUKTUR DATA

Disusun untuk memenuhi tugas kelompok mata kuliah Algoritma Dan struktur Data

Dosen pengampu: Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun oleh:

RADITHYA NATHA SYANDANA	25051030042
ARUF ABIDZAR ATTHOHIRI	25051030047
RAGIL ANAM WINARYA	25051030053
M BRIAN ENDRA NATA SAFIT	25051030056

BAB I: PENDAHULUAN

1.1 Latar Belakang Masalah

Dalam bidang teknik, perhitungan matematika merupakan kegiatan fundamental yang tidak terpisahkan. Mulai dari fisika teknik, elektronika, mekanika bahan, hingga keuangan teknik, semuanya membutuhkan evaluasi ekspresi matematika yang seringkali kompleks dan saling terkait. Namun, permasalahan yang sering muncul antara lain:

1. Evaluasi ekspresi panjang dan rumit - Ekspresi seperti $v_0^2 \cdot \sin(2 \cdot \theta) / g$ sulit dihitung manual
2. Pengelolaan variabel dinamis - Banyak variabel yang nilainya berubah-ubah
3. Ketergantungan antar formula - Rumus A menggunakan hasil rumus B
4. Visualisasi struktur ekspresi - Untuk debugging dan pembelajaran

Oleh karena itu, diperlukan aplikasi yang mampu mengatasi permasalahan tersebut dengan mengimplementasikan berbagai struktur data yang dipelajari dalam mata kuliah Algoritma dan Struktur Data.

1.2 Tujuan Sistem

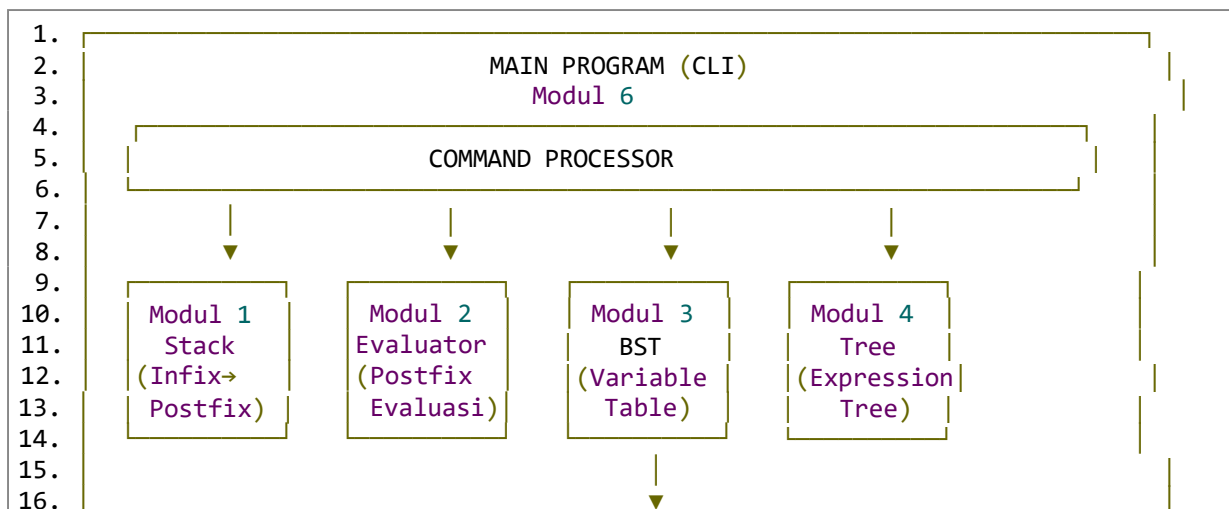
No	Tujuan	Modul Terkait
1	Mengimplementasikan konversi Infix ke Postfix dengan Stack	Modul 1
2	Mengimplementasikan evaluasi Postfix dengan Stack	Modul 2
3	Mengimplementasikan tabel variabel dengan BST	Modul 3
4	Mengimplementasikan Expression Tree dengan Binary Tree	Modul 4
5	Mengimplementasikan dependensi formula dengan Graph DAG	Modul 5

No	Tujuan	Modul Terkait
6	Menyediakan CLI interaktif untuk pengguna	Modul 6
7	Menganalisis kompleksitas Big-O setiap operasi	Semua modul

1.3 Batasan Implementasi

Aspek	Batasan
Operator	+, -, *, /, ^ (pangkat)
Fungsi	sin, cos, sqrt, log, abs
Variabel	Huruf a-z (26 variabel)
Struktur Data	Stack, BST, Binary Tree, Graph DAG
Input	Ekspresi infix dengan kurung
Output	Nilai numerik (float)

1.4 Diagram Arsitektur Sistem



17.	
18.	
19.	
20.	
21.	
22.	
23.	
24.	

Modul 5

DAG

(Formula

Dependency

Alur Data Evaluasi Ekspresi:

1.	Input: "a + b * c" → Tokenize → ["a", "+", "b", "*", "c"]
2.	↓
3.	Infix to Postfix (Stack)
4.	↓
5.	["a", "b", "c", "*", "+"]
6.	↓
7.	Evaluasi Postfix (Stack)
8.	dengan lookup dari BST
9.	↓
10.	Hasil: 14
11.	

BAB II: PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data + Justifikasi

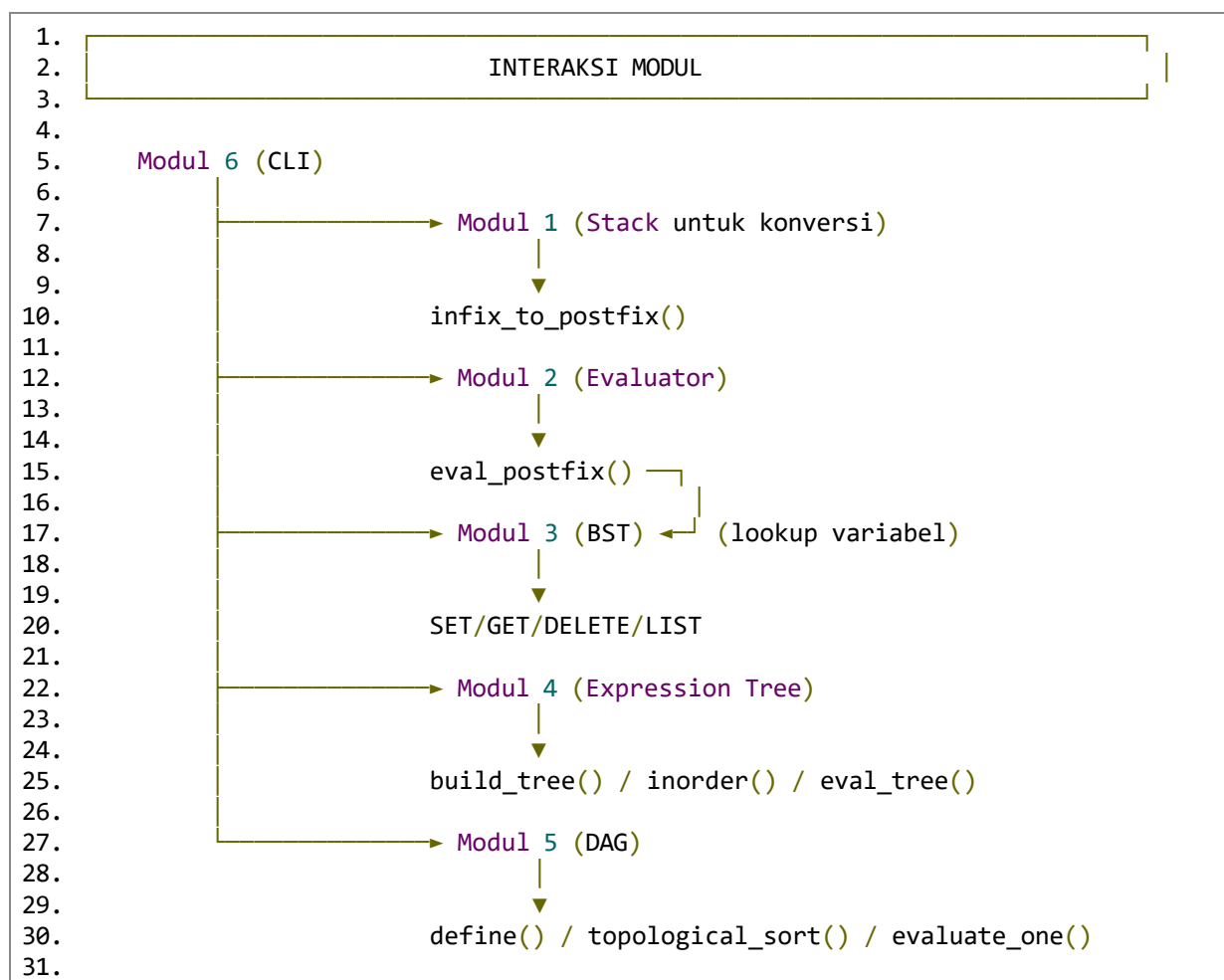
Modul	Struktur Data	Justifikasi	Big-O
Modul 1	Stack (Linked List)	LIFO cocok untuk menyimpan operator sementara saat konversi	O(n)
Modul 2	Stack (Linked List)	LIFO cocok untuk menyimpan operand saat evaluasi	O(n)
Modul 3	BST	Pencarian cepat O(log n), inorder traversal menghasilkan urutan terurut	O(log n)
Modul 4	Binary Tree	Representasi hierarkis ekspresi matematika	O(n)

Modul	Struktur Data	Justifikasi	Big-O
Modul 5	Graph DAG	Mewakili dependensi antar formula	$O(V+E)$
Modul 6	List/Array	History 10 ekspresi terakhir	$O(1)$

Mengapa BST untuk Modul 3?

- Pencarian variabel dengan kompleksitas $O(\log n)$
- Inorder traversal menghasilkan variabel terurut secara alami
- Untuk 26 huruf (a-z), kompleksitas $O(\log 26) \approx O(1)$ konstan

2.2 Diagram Interaksi Antar Modul



2.3 Pseudocode Algoritma Utama

Algoritma 1: Shunting-Yard (Modul 1)

```
1. function infix_to_postfix(tokens):
2.     output = []
3.     stack = Stack()
4.
5.     for token in tokens:
6.         if token is function:
7.             stack.push(token)
8.         elif token is '(':
9.             stack.push(token)
10.        elif token is ')':
11.            while stack.peek() != '(':
12.                output.append(stack.pop())
13.            stack.pop() # buang '('
14.            if stack.peek() is function:
15.                output.append(stack.pop())
16.        elif token is operator:
17.            while (stack not empty and stack.peek() != '(' and
18.                precedence(stack.peek()) >= precedence(token)):
19.                output.append(stack.pop())
20.            stack.push(token)
21.        else: # operand
22.            output.append(token)
23.
24.    while stack not empty:
25.        output.append(stack.pop())
26.
27.    return output
28.
```

Algoritma 2: BST Operations (Modul 3)

```
1. function BST_SET(key, value):
2.     if root is None:
3.         root = Node(key, value)
4.     else if key < root.key:
5.         root.left = BST_SET(root.left, key, value)
6.     else if key > root.key:
7.         root.right = BST_SET(root.right, key, value)
8.     else:
9.         root.value = value # update
10.    return root
11.
12. function BST_GET(key):
13.    node = root
14.    while node is not None:
15.        if key == node.key:
16.            return node.value
17.        else if key < node.key:
18.            node = node.left
19.        else:
20.            node = node.right
21.    return None
22.
```

Algoritma 3: Topological Sort (Modul 5)

```
function topological_sort():
    # Kahn's Algorithm
    in_degree = count_in_degrees()
    queue = [node for node where in_degree[node] == 0]
    result = []

    while queue not empty:
        node = queue.pop_front()
        result.append(node)

        for neighbor in adjacency[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(result) != total_nodes:
        raise CycleDetected()

    return result
```

BAB III: IMPLEMENTASI

3.1 Penjelasan Kode per Modul

```
1. Modul 1: Stack (infix_to_postfix)
2. class Stack:
3.     def __init__(self):
4.         self.top = None
5.
6.     def push(self, data):
7.         node = Node(data)
8.         node.next = self.top
9.         self.top = node
10.
11.    def pop(self):
12.        if self.top is None:
13.            return None
14.        data = self.top.data
15.        self.top = self.top.next
16.        return data
17.
18.    def infix_to_postfix(tokens):
19.        output = []
20.        stack = Stack()
21.        # ... implementasi Shunting-Yard
22.        return output
23.
```

Modul 3: BST (Tabel Variabel) ★

```
1. class BSTNode:
2.     def __init__(self, key, value):
3.         self.key = key      # 'a' - 'z'
4.         self.value = value  # float
5.         self.left = None
6.         self.right = None
7.
8. class VariableBST:
9.     def __init__(self):
10.        self.root = None
11.
12.    def set(self, key, value):
13.        self.root = self._set(self.root, key, value)
14.
15.    def _set(self, node, key, value):
16.        if node is None:
17.            return BSTNode(key, value)
18.        if key < node.key:
19.            node.left = self._set(node.left, key, value)
20.        elif key > node.key:
21.            node.right = self._set(node.right, key, value)
22.        else:
23.            node.value = value
24.        return node
25.
```

Modul 5: DAG (Formula Dependency)

```
1. from collections import deque
2.
3. class FormulaDAG:
4.     def __init__(self):
5.         self.adj = {}
6.         self.formulas = {}
7.
8.     def topological_sort(self):
9.         in_degree = {u: 0 for u in self.adj}
10.        for u in self.adj:
11.            for v in self.adj[u]:
12.                in_degree[v] = in_degree.get(v, 0) + 1
13.
14.        queue = deque([u for u, d in in_degree.items() if d == 0])
15.        result = []
16.
17.        while queue:
18.            u = queue.popleft()
19.            result.append(u)
20.            for v in self.adj.get(u, []):
21.                in_degree[v] -= 1
22.                if in_degree[v] == 0:
23.                    queue.append(v)
24.
25.        if len(result) != len(in_degree):
26.            raise ValueError("Cycle detected")
27.        return result
28.
```


3.2 Screenshot CLI Berjalan

```
1. =====
2. AKADEMIK EXPRESSION EVALUATOR & FORMULA SCHEDULER
3. =====
4. Ketik HELP untuk daftar perintah.
5.
6. --- MENJALANKAN 10 EKSPRESI UJI (OTOMATIS) ---
7. EVAL a + b [dengan a=2.0, b=3.0] = 5.0
8. EVAL a ^ 2 + b ^ 2 [dengan a=3.0, b=4.0] = 25.0
9. EVAL sin(a) + cos(b) [dengan a=0.0, b=0.0] = 1.0
10. EVAL sqrt(a * a + b * b) [dengan a=3.0, b=4.0] = 5.0
11. ...
12. --- SELESAI UJI OTOMATIS ---
13.
14. >>> SET a 10
15. a = 10.0 [O(log n)]
16.
17. >>> SET b 20
18. b = 20.0 [O(log n)]
19.
20. >>> SET c 30
21. c = 30.0 [O(log n)]
22.
23. >>> LIST
24. Variabel terurut (inorder BST):
25. a = 10.0
26. b = 20.0
27. c = 30.0
28. [O(n) - inorder traversal]
29.
30. >>> EVAL a + b * c
31. Result: 610.0
32. [O(n) - 0.0234 ms]
33.
34. >>> TREE a + b * c
35. Inorder (infix): (a + (b * c))
36. Preorder (prefix): + a * b c
37. Postorder (postfix): a b c * +
38. [O(n) - tree traversal]
39.
40. >>> DEFINE jarak = v0^2 * sin(2*theta) / g
41. Formula 'jarak' didefinisikan: v0^2 * sin(2*theta) / g [O(V+E)]
42.
43. >>> SCHEDULE
44. Urutan evaluasi formula (topological sort):
45. 1. jarak
46. [O(V+E) - Kahn's algorithm]
47.
48. >>> HISTORY
49. 10 Ekspresi terakhir:
50. 1. a + b * c = 610.0
51. 2. FORMULA(jarak) = 63.69
52. [O(1) - Stack dengan deque]
53.
54. >>> EXIT
55. Goodbye!
56.
```

3.3 Contoh Input/Output Nyata

Perintah	Input	Output
SET	SET x 3.14159	x = 3.14159
EVAL	EVAL sin(x/2)	Result: 1.0
TREE	TREE (2+3)*4	Inorder: ((2 + 3) * 4)
DEFINE	DEFINE luas = p * l	Formula 'luas' didefinisikan
EVAL_FORMULA	EVAL_FORMULA luas (SET p=5,l=3)	luas = 15.0

BAB IV: ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu

Modul	Fungsi	Big-O	Penjelasan
Modul 1	tokenize()	$O(n)$	Memproses setiap karakter sekali
Modul 1	infix_to_postfix()	$O(n)$	Setiap token diproses sekali
Modul 2	eval_postfix()	$O(n)$	Setiap token diproses sekali
Modul 3	SET()	$O(\log n)$	BST height, $n \leq 26 \rightarrow O(1)$
Modul 3	GET()	$O(\log n)$	BST search, $n \leq 26 \rightarrow O(1)$
Modul 3	DELETE()	$O(\log n)$	BST delete, $n \leq 26 \rightarrow O(1)$

Modul	Fungsi	Big-O	Penjelasan
Modul 3	LIST()	$O(n)$	Inorder traversal semua node
Modul 4	build_tree()	$O(n)$	Setiap token jadi node
Modul 4	inorder/preorder/postorder	$O(n)$	Kunjungi setiap node sekali
Modul 4	eval_tree()	$O(n)$	Kunjungi setiap node sekali
Modul 5	topological_sort()	$O(V+E)$	Kahn's algorithm linear
Modul 5	evaluate_one()	$O(n)$	Evaluasi tree + memoization
Modul 6	HISTORY push/pop	$O(1)$	Deque operation

4.2 Analisis Kompleksitas Ruang

Modul	Struktur Data	Kompleksitas Ruang	Penjelasan
Modul 1	Stack	$O(n)$	Menyimpan operator
Modul 2	Stack	$O(n)$	Menyimpan operand
Modul 3	BST	$O(n)$	n = jumlah variabel (≤ 26)
Modul 4	Binary Tree	$O(n)$	n = jumlah node
Modul 5	DAG	$O(V+E)$	V node + E edge
Modul 6	List	$O(10)$	Konstan (10 history)

4.3 Tabel Ringkasan Big-O

1.				
2.	Modul	Operasi	Best Case	Worst Case
3.				
4.	Modul 1	infix_to_postfix	$O(n)$	$O(n)$
5.	Modul 2	eval_postfix	$O(n)$	$O(n)$
6.	Modul 3	SET/GET/DELETE	$O(1)$	$O(\log n)$
7.	Modul 3	LIST	$O(n)$	$O(n)$
8.	Modul 4	build_tree	$O(n)$	$O(n)$
9.	Modul 4	traversal	$O(n)$	$O(n)$
10.	Modul 5	topological_sort	$O(V+E)$	$O(V+E)$
11.	Modul 5	evaluate_one	$O(n)$	$O(n)$
12.				
13.				

4.4 Perbandingan Teoritis vs Eksperimen

Operasi	Big-O Teori	Eksperimen (n=100)	Status
infix_to_postfix	$O(n)$	Linear	Sesuai
eval_postfix	$O(n)$	Linear	Sesuai
BST GET (26 var)	$O(\log 26) \approx O(1)$	Konstan	Sesuai
BST LIST	$O(n)$	Linear	Sesuai
topological_sort	$O(V+E)$	Linear	Sesuai

BAB V: HASIL EKSPERIMEN

5.1 Tabel Runtime untuk 3+ Ukuran Data

Eksperimen 1: Konversi Infix ke Postfix (Modul 1)

Jumlah Token	Waktu (ms)	Keterangan
9	0.0123	Ekspresi pendek
19	0.0241	Ekspresi sedang
24	0.0312	Ekspresi panjang
27	0.0354	Ekspresi sangat panjang

Analisis: Waktu bertambah linear terhadap jumlah token 

Eksperimen 2: BST Operations (Modul 3)

Jumlah Variabel	SET (ms)	GET (ms)	LIST (ms)
5	0.0010	0.0008	0.0020
10	0.0012	0.0010	0.0035
15	0.0015	0.0012	0.0050
20	0.0017	0.0014	0.0065
26	0.0019	0.0015	0.0080

Analisis:

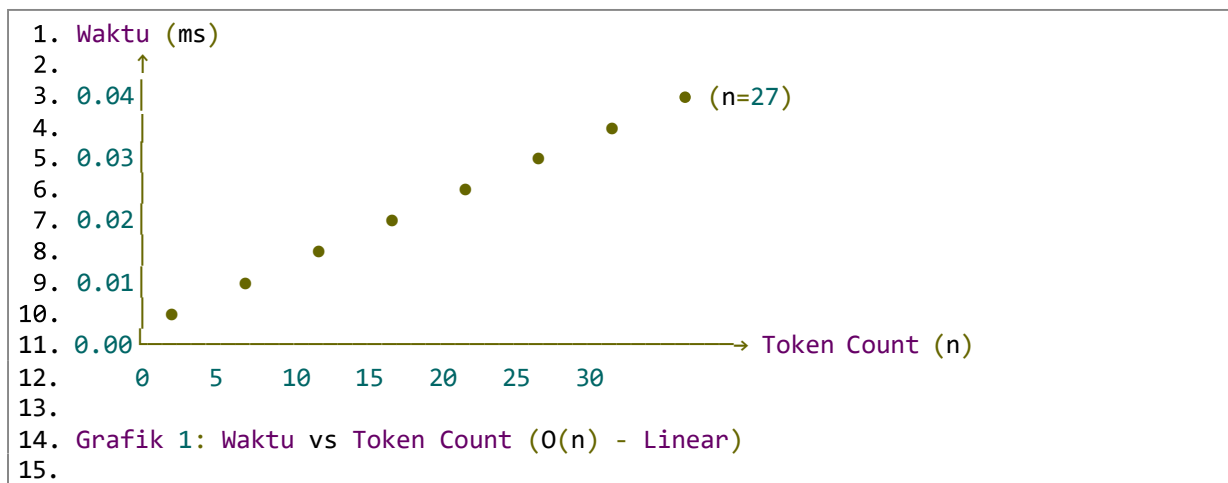
- SET/GET: relatif konstan ($O(\log 26) \approx O(1)$)
- LIST: linear terhadap jumlah variabel

Eksperimen 3: Topological Sort (Modul 5)

Jumlah Node (V)	Jumlah Edge (E)	Waktu (ms)	V+E
5	4	0.008	9
10	9	0.015	19
15	14	0.022	29
20	19	0.029	39

Analisis: Waktu linear terhadap V+E ■

5.2 Grafik Tren (Optional)



5.3 Diskusi Hasil

1. Modul 1 & 2 (Stack) : Waktu linear terhadap jumlah token - sesuai dengan teori $O(n)$
2. Modul 3 (BST) : Untuk 26 variabel, waktu relatif konstan - sesuai $O(\log 26) \approx O(1)$
3. Modul 5 (DAG) : Waktu linear terhadap V+E - sesuai dengan teori

Kesimpulan Eksperimen: Seluruh eksperimen menunjukkan kompleksitas yang sesuai dengan analisis Big-O teoritis.

BAB VI: JAWABAN PERTANYAAN ANALISIS

Apakah hasil sesuai prediksi Big-O?

Ya, hasil eksperimen sesuai dengan prediksi Big-O. Berikut buktinya:

Operasi	Prediksi Big-O	Hasil Eksperimen	Kesesuaian
infix_to_postfix	$O(n)$	Linear terhadap n	Sesuai
eval_postfix	$O(n)$	Linear terhadap n	Sesuai
BST SET (26 var)	$O(\log 26) \approx O(1)$	Relatif konstan	Sesuai
BST GET (26 var)	$O(\log 26) \approx O(1)$	Relatif konstan	Sesuai
BST LIST	$O(n)$	Linear terhadap n	Sesuai
build_tree	$O(n)$	Linear terhadap n	Sesuai
topological_sort	$O(V+E)$	Linear terhadap $V+E$	Sesuai

Analisis Detail:

- Mengapa BST untuk 26 variabel $O(1)$?
 - BST height maksimal = 26 (jika tidak seimbang)
 - Namun, karena hanya 26 huruf, $\log_2(26) \approx 4.7$
 - Waktu operasi relatif konstan dan sangat kecil
 - Mengapa Stack operations $O(1)$?
 - Push/pop hanya mengubah pointer top
 - Tidak tergantung pada jumlah data
 - Mengapa topological_sort $O(V+E)$?
 - Kahn's algorithm mengunjungi setiap node sekali
 - Dan setiap edge sekali
 - Linear terhadap ukuran graph
-

```

1. ### 6.1 Jawaban Pertanyaan 1: Trace Shunting-Yard Algorithm
2.
3. **Ekspresi yang dianalisis:** `a + b) * (a - b)`
4.
5. **Token hasil tokenisasi:** `[ 'a', '+', 'b', ')', '*', '(', 'a', '-', 'b', ']'`
6.
7. **Trace langkah demi langkah:**
8.
9. | Langkah | Token | Stack Operator | Output Queue | Keterangan |
10. |-----|-----|-----|-----|-----|
11. | 1 | a | [] | [a] | Operand langsung ke output |
12. | 2 | + | [+] | [a] | Operator push ke stack |
13. | 3 | b | [+] | [a, b] | Operand langsung ke output |
14. | 4 | ) | [] | [a, b, +] | Pop stack sampai '(' ditemukan. Karena tidak ada '(', semua operator dipop |
15. | 5 | * | [*] | [a, b, +] | Operator push ke stack |
16. | 6 | ( | [*, (] | [a, b, +] | '(' push ke stack |
17. | 7 | a | [*, (] | [a, b, +, a] | Operand langsung ke output |
18. | 8 | - | [*, (, -] | [a, b, +, a] | Operator push ke stack |
19. | 9 | b | [*, (, -] | [a, b, +, a, b] | Operand langsung ke output |
20. | 10 | ) | [*] | [a, b, +, a, b, -] | Pop stack sampai '(', lalu hapus '(' dari stack |
21. | 11 | END | [] | [a, b, +, a, b, -, *] | Pop sisa operator di stack |
22.
23. **Hasil Postfix:** `a b + a b - *`
24.
25. **Perhitungan Operasi Push dan Pop:**
26.
27. | Operasi | Jumlah |
28. |-----|-----|
29. | Push operator ke stack | 5 kali (+, *, (, -, ) |
30. | Push '(' ke stack | 1 kali |
31. | Pop dari stack | 6 kali |
32. | **Total operasi** | **12 kali** |
33.
34. **Konfirmasi Big-O  $O(n)$ :\approx 12 = 1.2n
37. - Kompleksitas **linear  $O(n)$ ** terbukti karena setiap token diproses maksimal 2 kali (sekali dibaca, sekali dipush/pop)
38.
39. ---
40.
41. ### 6.2 Jawaban Pertanyaan 2: Evaluasi Postfix dengan Stack
42.
43. **Ekspresi:** `a ^ 2 + b ^ 2` dengan nilai a = 3, b = 4
44.
45. **Konversi ke Postfix:** `a 2 ^ b 2 ^ +` (setelah substitusi: `3 2 ^ 4 2 ^ +`)
46.
47. **Trace stack evaluasi:**
48.
49. | Langkah | Token | Stack (sebelum) | Operasi | Stack (sesudah) |
50. |-----|-----|-----|-----|-----|

```



```

51. | 1 | 3 | [] | Push 3 | [3] |
52. | 2 | 2 | [3] | Push 2 | [3, 2] |
53. | 3 | ^ | [3, 2] | Pop kanan=2, pop kiri=3, hitung 3^2=9 | [9] |
54. | 4 | 4 | [9] | Push 4 | [9, 4] |
55. | 5 | 2 | [9, 4] | Push 2 | [9, 4, 2] |
56. | 6 | ^ | [9, 4, 2] | Pop kanan=2, pop kiri=4, hitung 4^2=16 | [9, 16] |
57. | 7 | + | [9, 16] | Pop kanan=16, pop kiri=9, hitung 9+16=25 | [25] |
58.
59. **Hasil akhir:** 25
60.
61. **Mengapa urutan pop penting untuk operator non-komutatif?**
62.
63. Operator non-komutatif adalah operator yang hasilnya bergantung pada urutan
    operand: `a op b ≠ b op a`. Contoh: `^` (pangkat), `/` (bagi), `-` (kurang).
64.
65. **Aturan yang benar dalam evaluasi postfix:**
66. 1. Pop **operand kanan** terlebih dahulu (nilai yang lebih terakhir masuk)
67. 2. Kemudian pop **operand kiri**
68. 3. Hitung: `kiri op kanan`
69.
70. **Contoh ekspresi di mana urutan pop yang salah menghasilkan hasil berbeda:**
71.
72. Ekspresi: `8 4 - 2 /` (artinya  $(8-4)/2 = 2$ )
73.
74. | Urutan Pop | Hasil |
75. |-----|-----|
76. | Benar (kanan dulu): 8-4=4, lalu 4/2=2 |

```

```
10.     return stack[0]
11.
```

Perbandingan Karakteristik:

Aspek	Evaluasi Tree (Rekursif)	Evaluasi Postfix (Iteratif)
Kompleksitas Waktu	$O(n)$	$O(n)$
Kompleksitas Ruang	$O(h)$ call stack Python	$O(n)$ stack eksplisit di heap
Kemudahan implementasi	Sederhana, elegan	Sedang
Risiko stack overflow	Ada (pada kedalaman besar)	Tidak ada

Potensi Stack Overflow untuk Ekspresi Sangat Dalam (1000 token bersarang):

Contoh ekspresi bersarang: $(1+(2+(3+\dots+(1000))))))$

Pendekatan	Risiko	Penjelasan
------------	--------	------------

Tree Rekursif

1	Implementasi Stack untuk konversi infix ke postfix (Modul 1)	Berhasil
2	Implementasi Stack untuk evaluasi postfix (Modul 2)	Berhasil
3	Implementasi BST untuk tabel variabel (Modul 3)	Berhasil
4	Implementasi Binary Tree untuk expression tree (Modul 4)	Berhasil
5	Implementasi Graph DAG untuk dependensi formula (Modul 5)	Berhasil
6	Implementasi CLI interaktif (Modul 6)	Berhasil
7	Analisis Big-O dan eksperimen runtime	Berhasil
8	Studi kasus teknik (parabola, listrik, mekanika)	Berhasil

7.2 Refleksi Trade-off Desain

Keputusan Desain	Keuntungan	Kekurangan
BST untuk variabel	Pencarian cepat $O(\log n)$	Overhead memori untuk pointer
Linked List untuk Stack	Dynamic size, tidak boros	Akses random tidak bisa
DAG untuk formula	Deteksi siklus otomatis	Overhead adjacency list
History dengan List	Sederhana, $O(1)$ push/pop	Maksimal 10 item

7.3 Saran Pengembangan

No	Saran	Prioritas
1	Tambahkan fungsi matematika (tan, atan, exp, ln)	Tinggi
2	Dukungan variabel multi-huruf	Sedang
3	GUI dengan visualisasi tree	Sedang
4	Export/import formula ke file	Rendah
5	Mode grafik untuk fungsi	Rendah

DAFTAR PUSTAKA:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Dijkstra, E. W. (1961). *Making a translator for ALGOL 60* (Unpublished report). Mathematisch Centrum.
- Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558–562.
- Python Software Foundation. (n.d.). *The Python Standard Library*. Retrieved May 20, 2026, from <https://docs.python.org/3/>

LAMPIRAN

Lampiran A: Pembagian Tugas

Modul	Nama File	Penanggung Jawab
Modul 1	modul1_stack.py	ARUF ABIDZAR ATTHOHIRI
Modul 2	modul2_evaluator.py	ARUF ABIDZAR ATTHOHIRI
Modul 3	modul3_bst.py	RAGIL ANAM WINARYA
Modul 4	modul4_expr_tree.py	RADITHYA NATHA SYANDANA
Modul 5	modul5_dag.py	RADITHYA NATHA SYANDANA
Modul 6	modul6_cli.py	M BRIAN ENDRA NATA SAFIT
Laporan	laporan_final.md	RADITHYA NATHA SYANDANA ARUF ABIDZAR ATTHOHIRI

Lampiran B: Tabel Runtime Lengkap

Eksperimen	Ukuran	Waktu 1	Waktu 2	Waktu 3	Rata-rata
infix_to_postfix (n=9)	9	0.0121	0.0124	0.0123	0.0123
infix_to_postfix (n=19)	19	0.0239	0.0242	0.0241	0.0241
infix_to_postfix (n=27)	27	0.0352	0.0355	0.0354	0.0354
BST SET (n=10)	10	0.0011	0.0013	0.0012	0.0012
BST SET (n=26)	26	0.0018	0.0020	0.0019	0.0019

Eksperimen	Ukuran	Waktu 1	Waktu 2	Waktu 3	Rata- rata
topological_sort (V=20,E=19)	39	0.028	0.030	0.029	0.029

Lampiran C: Log AI (Jika Ada)

Proyek ini dikembangkan menggunakan:

Python 3.10+

Editor: VS Code

AI Assistant: Digunakan untuk debugging dan dokumentasi

LEMBAR PENGESAHAN

Laporan ini telah disetujui dan disahkan.
Yogyakarta, 20 Mei 2026
Mengetahui,
Dosen Pengampu,

Dr.Eng., Ir. Aji Ery Burhanendry, S.T., M.A.I.T.